ToptanNext — Agent Mimari & Geliştirme Kılavuzu

> Bu dosya, ToptanNext projesini geliştiren her AI ajanının (Cursor, Copilot, Claude Code vb.) okuyup içselleştirmesi gereken **tek kaynak belgedir**. Kod yazmadan önce bu dosyayı baştan sona oku. Karşılaştığın her mimari kararda buraya dön.

---

1. Proje Kimliği

Alan	Değer
Proje Adı	ToptanNext
Tip	B2B Toptan Satış Pazaryeri (Alibaba benzeri)
Hedef Pazar	Türkiye
Dil	Türkçe arayüz, İngilizce kod ve değişken adları
Ödeme	iyzico (Sandbox → Production)
Ölçek Hedefi	Yüzlerce tedarikçi, binlerce alıcı, yüz binlerce ürün

Kullanıcı Rolleri

```
ADMIN      → Platformu yönetir. Tüm verilere erişir.
SUPPLIER   → Tedarikçi. Ürün ekler, siparişleri yönetir, mağaza sahibidir.
BUYER      → Alıcı. Ürünlere göz atar, sipariş verir, fatura alır.
```

---

2. Monorepo Yapısı

Bu proje **Turborepo** ile yönetilen bir monorepo'dur. Hiçbir zaman `apps/` veya `packages/` dışına kod yazma.

```
toptannext/
├── apps/
│   ├── web/          → Next.js 14 — Alıcı & tedarikçi arayüzü (SSR, SEO kritik)
│   ├── mobile/       → Expo (React Native) — iOS & Android
│   ├── admin/        → Next.js 14 — Sadece ADMIN rolü için yönetim paneli
│   └── api/          → NestJS — Tüm backend servisleri
│
├── packages/
│   ├── ui/           → Ortak UI bileşenleri (web + admin için)
│   ├── types/        → Paylaşılan TypeScript tipleri ve Zod şemaları
│   ├── config/       → ESLint, Prettier, TypeScript base config
│   └── utils/        → Platform-agnostic yardımcı fonksiyonlar
│
├── turbo.json
├── package.json      → Root workspace tanımı
└── MANA.md           → Bu dosya
```

Kritik Kural

`packages/types` içindeki tipler **tek kaynak** (single source of truth) dir. Aynı tipi `apps/api` ve `apps/web` içinde ayrı ayrı tanımlama. Her zaman `@toptannext/types` paketinden import et.

---

3. Teknoloji Yığını (Stack)

3.1 Frontend — `apps/web` ve `apps/admin`

Teknoloji	Versiyon	Amaç
Next.js	14+ (App Router)	SSR/SSG, SEO, routing
TypeScript	5+	Tip güvenliği
Tailwind CSS	3+	Stil sistemi
TanStack Query	5+	Server-state, cache, optimistic updates
TanStack Table	8+	Admin paneli veri tabloları
React Hook Form	7+	Form yönetimi
Zod	3+	Form & API validasyonu
Lucide React	latest	İkon seti
next-intl	3+	i18n (Türkçe öncelikli)

Next.js App Router kuralları:

- `page.tsx` → Sadece route tanımı, veri fetch server component'ta yapılır.
- `layout.tsx` → Ortak layout, auth kontrolü burada yapılır.
- `loading.tsx` → Skeleton UI her route için zorunlu.
- `error.tsx` → Her route segmentinde hata sınırı olmalı.
- `(group)/` klasörleri → URL'yi etkilemeyen mantıksal gruplama için kullan.
- `_components/` → O sayfaya özel, dışarıya export edilmeyecek bileşenler.

3.2 Mobile — `apps/mobile`

Teknoloji	Versiyon	Amaç
Expo	SDK 51+	Cross-platform mobile
Expo Router	3+	File-based routing
NativeWind	4+	Tailwind for React Native
TanStack Query	5+	Aynı API hook'ları web ile paylaşılır

3.3 Backend — `apps/api`

Teknoloji	Versiyon	Amaç
NestJS	10+	Modüler backend framework
TypeScript	5+	Tip güvenliği
Prisma	5+	ORM, database schema
class-validator	latest	DTO validasyonu
class-transformer	latest	DTO dönüşümü
Passport.js	latest	Auth stratejileri
JWT	latest	Token yönetimi
BullMQ	latest	Job queue (async işlemler)
Socket.io	latest	WebSocket (real-time bildirimler)

3.4 Veri Katmanı

Teknoloji	Amaç
PostgreSQL 16	Ana veritabanı — ACID, finansal işlemler
Redis 7	Cache + BullMQ job queue backend
Meilisearch	Ürün arama motoru — hızlı, Türkçe dil desteği
AWS S3	Ürün görselleri ve belgeler (vergi levhası vb.)
AWS CloudFront	CDN — görseller için

3.5 Altyapı & DevOps

Teknoloji	Amaç
Docker + Docker Compose	Local development ortamı
Nginx	Reverse proxy, rate limiting, SSL termination
GitHub Actions	CI/CD pipeline
Sentry	Hata izleme (hem frontend hem backend)
Resend	Transactional email (sipariş onayı, fatura vb.)

---

4. Backend Mimari Detayları (`apps/api`)

4.1 NestJS Modül Yapısı

Her domain kendi modülüdür. Modüller arası doğrudan bağımlılık **yasaktır** — sadece servis injection ile iletişim kurulur.

```
apps/api/src/
├── main.ts
├── app.module.ts
│
├── modules/
│   ├── auth/               → JWT, Passport, login/register
│   ├── users/              → Kullanıcı profili, RBAC
│   ├── suppliers/          → Tedarikçi mağazası, onay süreci
│   ├── products/           → Ürün CRUD, kategori, varyant
│   ├── inventory/          → Stok yönetimi, stok hareketleri
│   ├── orders/             → Sipariş yaşam döngüsü
│   ├── payments/           → iyzico entegrasyonu, webhook
│   ├── cart/               → Sepet (Redis'te tutulur)
│   ├── search/             → Meilisearch senkronizasyonu
│   ├── notifications/      → Email, push, WebSocket bildirimleri
│   └── admin/              → Admin-only endpoint'ler
│
├── common/
│   ├── decorators/         → @CurrentUser(), @Roles(), @Public()
│   ├── guards/             → JwtAuthGuard, RolesGuard
│   ├── filters/            → GlobalExceptionFilter
│   ├── interceptors/       → LoggingInterceptor, TransformInterceptor
│   ├── pipes/              → ValidationPipe config
│   └── types/              → Backend-specific tipler
│
├── config/
│   ├── database.config.ts
│   ├── redis.config.ts
│   ├── jwt.config.ts
│   └── storage.config.ts
│
└── prisma/
    ├── schema.prisma
    └── migrations/
```

4.2 Her Modülün İç Yapısı

```
modules/products/
├── products.module.ts
├── products.controller.ts    → HTTP endpoint'leri, sadece DTO al/ver
├── products.service.ts       → İş mantığı burada
├── products.repository.ts    → Prisma sorguları burada (controller'a Prisma
import etme)
├── dto/
│   ├── create-product.dto.ts
```

```
|   ├── update-product.dto.ts
|   └── product-filter.dto.ts
└── entities/
    └── product.entity.ts     → Dışarıya expose edilecek tip (password gibi
alanlar hariç)
```

4.3 API Endpoint Konvansiyonu

```
GET    /api/v1/products              → Liste (sayfalı)
GET    /api/v1/products/:id          → Tek kayıt
POST   /api/v1/products              → Oluştur
PATCH  /api/v1/products/:id          → Kısmi güncelle
DELETE /api/v1/products/:id          → Sil

GET    /api/v1/suppliers/:id/products → İlişkili kayıtlar
POST   /api/v1/orders/:id/cancel      → Aksiyon endpoint'leri
```

Her endpoint:

- DTO ile valide edilir (`class-validator`)
- `@Roles()` decorator ile korunur
- Standart response formatı döner (aşağıya bak)
- Hata durumunda `GlobalExceptionFilter` yakalar

4.4 Standart API Response Formatı

```typescript
```

```
// Başarılı
{
  "success": true,
  "data": { ... },
  "meta": {              // Sayfalı listeler için
    "page": 1,
    "limit": 20,
    "total": 450,
    "totalPages": 23
  }
}

// Hata
{
  "success": false,
  "error": {
    "code": "PRODUCT_NOT_FOUND",
    "message": "Ürün bulunamadı",
    "details": []         // Validasyon hataları için
  }
}
```

4.5 RBAC (Role-Based Access Control)

```typescript
// Kullanım
@Get('admin/stats')
@Roles(Role.ADMIN)
getStats() { ... }

@Post('products')
@Roles(Role.SUPPLIER, Role.ADMIN)
createProduct() { ... }

@Get('products')
@Public()  // JWT gerektirmez
listProducts() { ... }
```

---

5. Veritabanı Şeması (Prisma)

5.1 Temel Modeller

```prisma
```

```
model User {
  id            String    @id @default(cuid())
  email         String    @unique
  passwordHash  String
  role          Role      @default(BUYER)
  isVerified    Boolean   @default(false)
  isActive      Boolean   @default(true)
  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt

  profile       Profile?
  supplier      Supplier?
  orders        Order[]
  addresses     Address[]
}

model Supplier {
  id            String    @id @default(cuid())
  userId        String    @unique
  companyName   String
  taxNumber     String    @unique
  taxOffice     String
  isApproved    Boolean   @default(false)
  commissionRate Decimal  @default(0.05)  // %5
  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt

  user          User      @relation(fields: [userId], references: [id])
  products      Product[]
  orders        OrderItem[]
}

model Product {
  id            String    @id @default(cuid())
  supplierId    String
  categoryId    String
  name          String
  slug          String    @unique
  description   String    @db.Text
  basePrice     Decimal
  currency      String    @default("TRY")
  minOrderQty   Int       @default(1)
  unit          String    // "adet", "kg", "metre" vb.
  isActive      Boolean   @default(true)
  isApproved    Boolean   @default(false)
  createdAt     DateTime  @default(now())
```

```
  updatedAt     DateTime  @updatedAt

  supplier      Supplier  @relation(fields: [supplierId], references: [id])
  category      Category  @relation(fields: [categoryId], references: [id])
  variants      ProductVariant[]
  images        ProductImage[]
  inventory     Inventory?
}

model Order {
  id            String      @id @default(cuid())
  buyerId       String
  status        OrderStatus @default(PENDING)
  totalAmount   Decimal
  currency      String      @default("TRY")
  paymentId     String?     // iyzico payment ID
  paymentStatus PaymentStatus @default(WAITING)
  shippingAddressId String
  notes         String?
  createdAt     DateTime    @default(now())
  updatedAt     DateTime    @updatedAt

  buyer         User        @relation(fields: [buyerId], references: [id])
  items         OrderItem[]
  payment       Payment?
  shippingAddress Address   @relation(fields: [shippingAddressId], references: [id])
}

enum Role            { ADMIN SUPPLIER BUYER }
enum OrderStatus     { PENDING CONFIRMED PROCESSING SHIPPED DELIVERED CANCELLED REFU
enum PaymentStatus   { WAITING SUCCESS FAILED REFUNDED }
```

5.2 Prisma Kullanım Kuralları

- Repository katmanına Prisma'yı enjekte et, service veya controller'da `prisma.xxx` çağırma.
- `select` kullanarak sadece ihtiyaç duyulan alanları çek — `findMany` ile tüm alanları çekme.
- Finansal işlemlerde her zaman `prisma.$transaction([...])` kullan.
- Hassas alanlar (`passwordHash`, `taxNumber`) hiçbir zaman response'a dahil edilmez.

---

6. Ödeme Akışı (iyzico)

```
1. Alıcı "Ödemeyi Tamamla" butonuna tıklar
   └─ POST /api/v1/payments/initialize
      └─ iyzico Checkout Form token alınır
      └─ Order PENDING olarak oluşturulur

2. Frontend iyzico iFrame'i render eder
   └─ Kullanıcı kart bilgisini girer (PCI-DSS: biz kart görmeyiz)

3. iyzico ödemeyi işler
   └─ POST /api/v1/payments/webhook  ← iyzico buraya callback atar
      └─ İmza doğrulanır (HMAC)
      └─ Başarılıysa:
         - Order status → CONFIRMED
         - Stok düşülür (Inventory)
         - Tedarikçiye bildirim gönderilir (BullMQ job)
         - Alıcıya onay emaili gönderilir (BullMQ job)
      └─ Başarısızsa:
         - Order status → CANCELLED
         - Alıcıya hata emaili gönderilir

4. Frontend sonuç sayfasını gösterir
   └─ GET /api/v1/orders/:id ile sipariş durumu sorgulanır
```

Webhook endpoint'i:

- Public endpoint (JWT gerektirmez)
- iyzico imzası doğrulanmadan hiçbir işlem yapılmaz
- İdempotent olmalı — aynı webhook 2 kez gelirse 2 kez işlenmemeli

---

7. Async Job'lar (BullMQ)

Redis üzerinde çalışan queue sistemi. Ağır ve gecikebilir işlemleri ana request döngüsünden çıkar.

```
Queue: "emails"
  ├── order-confirmation     → Sipariş onayı emaili
  ├── order-shipped          → Kargo bildirimi
  ├── supplier-new-order     → Tedarikçiye yeni sipariş bildirimi
  ├── password-reset         → Şifre sıfırlama
  └── welcome                → Hoş geldin emaili

Queue: "inventory"
  └── sync-to-search         → Stok değişikliğini Meilisearch'e senkronize et

Queue: "notifications"
  └── push-notification      → Mobile push bildirimi

Queue: "reports"
  └── generate-invoice       → PDF fatura oluştur, S3'e yükle
```

Kural: Job'lar her zaman idempotent yazılır. Job başarısız olursa retry mekanizması ile tekrar çalışır.

---

8. Arama Sistemi (Meilisearch)

Meilisearch, PostgreSQL'in kopyasını (replica) tutar — sadece arama için kullanılır, asıl veri PostgreSQL'dedir.

```typescript
// Index: "products"
// Searchable fields:
//   name, description, category.name, supplier.companyName
// Filterable fields:
//   categoryId, supplierId, minOrderQty, currency, isActive
// Sortable fields:
//   basePrice, createdAt
// Facets:
//   categoryId, supplierId

// Senkronizasyon tetikleyicileri:
// - Ürün oluşturulduğunda (BullMQ job)
// - Ürün güncellendiğinde (BullMQ job)
// - Ürün silindiğinde (BullMQ job)
// - Stok değiştiğinde (BullMQ job)
```

---

9. Real-time Bildirimler (Socket.io)

```typescript
// Namespace: /notifications

// Tedarikçi bağlandığında:
socket.join(`supplier:${supplierId}`)

// Alıcı bağlandığında:
socket.join(`buyer:${buyerId}`)

// Olaylar:
"order:new"         → Tedarikçiye: yeni sipariş geldi
"order:status"      → Alıcıya: siparişin durumu değişti
"stock:low"         → Tedarikçiye: stok kritik seviyeye düştü
```

---

10. Frontend Mimari — `apps/web`

10.1 Klasör Yapısı

```
apps/web/src/
├── app/
│   ├── (public)/               → Auth gerektirmeyen sayfalar
│   │   ├── page.tsx            → Ana sayfa
│   │   ├── products/
│   │   │   ├── page.tsx        → Ürün listesi (SSR)
│   │   │   └── [slug]/
│   │   │       └── page.tsx    → Ürün detay (SSG + ISR)
│   │   ├── suppliers/
│   │   │   └── [id]/page.tsx   → Mağaza sayfası
│   │   └── search/
│   │       └── page.tsx        → Arama sonuçları
│   │
│   ├── (auth)/                 → Login, register, şifre sıfırlama
│   │   ├── login/page.tsx
│   │   └── register/page.tsx
│   │
│   ├── (buyer)/                → BUYER rolü — auth gerekli
│   │   ├── dashboard/page.tsx
│   │   ├── orders/page.tsx
│   │   ├── cart/page.tsx
│   │   └── profile/page.tsx
│   │
```

```
│   ├── (supplier)/             → SUPPLIER rolü — auth gerekli
│   │   ├── dashboard/page.tsx
│   │   ├── products/page.tsx
│   │   ├── orders/page.tsx
│   │   └── store/page.tsx
│   │
│   ├── layout.tsx              → Root layout (font, provider'lar)
│   └── globals.css
│
├── components/
│   ├── ui/                     → Temel bileşenler (Button, Input, Modal)
│   ├── product/                → ProductCard, ProductGrid, ProductFilter
│   ├── order/                  → OrderCard, OrderTimeline
│   ├── cart/                   → CartItem, CartSummary
│   └── layout/                 → Navbar, Footer, Sidebar
│
├── hooks/
│   ├── use-auth.ts
│   ├── use-cart.ts
│   └── use-products.ts
│
├── lib/
│   ├── api.ts                  → Axios instance, interceptor'lar
│   ├── query-client.ts         → TanStack Query config
│   └── utils.ts
│
└── middleware.ts                → Auth kontrolü, role-based redirect
```

10.2 Data Fetching Kuralları

```typescript
```

```
// Server Component — SEO kritik sayfalar
// app/(public)/products/[slug]/page.tsx
export default async function ProductPage({ params }) {
  const product = await fetch(`${API}/products/${params.slug}`, {
    next: { revalidate: 3600 }  // ISR: saatte bir yenile
  });
  return <ProductDetail product={product} />;
}

// Client Component — interaktif liste, filtre, arama
// components/product/ProductGrid.tsx
'use client';
const { data, isLoading } = useQuery({
  queryKey: ['products', filters],
  queryFn: () => api.get('/products', { params: filters }),
});
```

10.3 SEO Kuralları

Her `page.tsx` mutlaka `generateMetadata` export eder:

```typescript
export async function generateMetadata({ params }): Promise<Metadata> {
  const product = await getProduct(params.slug);
  return {
    title: `${product.name} | ToptanNext`,
    description: product.description.substring(0, 160),
    openGraph: {
      images: [product.images[0]?.url],
    },
  };
}
```

Ürün listesi ve detay sayfaları **kesinlikle** SSR/ISR kullanır — client-only render yasak.

---

11. Kod Konvansiyonları

11.1 Genel Kurallar

- **Dil:** Değişken, fonksiyon, class isimleri İngilizce. Yorum satırları Türkçe olabilir.
- **Tip güvenliği:** `any` kullanımı yasak. `unknown` + type guard tercih edilir.
- **`console.log`:** Production kodunda yasak. `Logger` servisi kullan.
- **Magic number:** Sayısal sabitler için enum veya constant kullan.

- **Dosya adı:** `kebab-case.ts` (NestJS) ve `PascalCase.tsx` (React bileşenleri).

11.2 TypeScript

```typescript
// ❌ Yapma
const processOrder = (data: any) => { ... }

// ✅ Yap
const processOrder = (data: CreateOrderDto): Promise<OrderEntity> => { ... }

// ❌ Yapma
const user = users.find(u => u.id === id)!  // non-null assertion

// ✅ Yap
const user = users.find(u => u.id === id);
if (!user) throw new NotFoundException(`Kullanıcı bulunamadı: ${id}`);
```

11.3 NestJS DTO Örneği

```typescript
import { IsString, IsNumber, Min, IsOptional } from 'class-validator';
import { Transform } from 'class-transformer';

export class CreateProductDto {
  @IsString()
  name: string;

  @IsNumber()
  @Min(0)
  @Transform({ value }) => parseFloat(value))
  basePrice: number;

  @IsNumber()
  @Min(1)
  minOrderQty: number;

  @IsOptional()
  @IsString()
  description?: string;
}
```

11.4 React Bileşen Örneği

```typescript
```

```
// ✅ Doğru bileşen yapısı
interface ProductCardProps {
  product: ProductEntity;
  onAddToCart?: (productId: string, qty: number) => void;
}

export function ProductCard({ product, onAddToCart }: ProductCardProps) {
  // ...
}

// ❌ Default export bileşeni dışa aktarma (named export tercih edilir)
// export default function ProductCard() { ... }
```

---

12. Ortam Değişkenleri

```
env
```

```
# apps/api/.env
DATABASE_URL="postgresql://user:pass@localhost:5432/toptannext"
REDIS_URL="redis://localhost:6379"
MEILISEARCH_URL="http://localhost:7700"
MEILISEARCH_MASTER_KEY="..."

JWT_SECRET="..."
JWT_EXPIRES_IN="7d"
JWT_REFRESH_EXPIRES_IN="30d"

IYZICO_API_KEY="..."
IYZICO_SECRET_KEY="..."
IYZICO_BASE_URL="https://sandbox-api.iyzipay.com"
IYZICO_WEBHOOK_SECRET="..."

AWS_REGION="eu-central-1"
AWS_ACCESS_KEY_ID="..."
AWS_SECRET_ACCESS_KEY="..."
S3_BUCKET_NAME="toptannext-media"
CLOUDFRONT_URL="https://cdn.toptannext.com"

RESEND_API_KEY="..."
SENTRY_DSN="..."

# apps/web/.env.local
NEXT_PUBLIC_API_URL="http://localhost:3001/api/v1"
NEXT_PUBLIC_SOCKET_URL="http://localhost:3001"
NEXT_PUBLIC_SENTRY_DSN="..."
NEXT_PUBLIC_MEILISEARCH_URL="http://localhost:7700"
NEXT_PUBLIC_MEILISEARCH_SEARCH_KEY="..."  # Sadece arama izni olan key
```

---

13. Docker Local Geliştirme

yaml

```yaml
# docker-compose.yml — sadece infrastructure servisleri
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: toptannext
    ports: ["5432:5432"]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  meilisearch:
    image: getmeili/meilisearch:latest
    ports: ["7700:7700"]
```

Uygulamalar Docker içinde değil, local'de çalışır (HMR için):

```bash
# Terminal 1: Infrastructure
docker-compose up

# Terminal 2: API
cd apps/api && pnpm dev

# Terminal 3: Web
cd apps/web && pnpm dev

# Terminal 4: Admin
cd apps/admin && pnpm dev
```

---

14. Güvenlik Kontrol Listesi

- [ ] Her endpoint'te `@Roles()` veya `@Public()` decorator'ı var
- [ ] Finansal işlemler Prisma transaction içinde
- [ ] iyzico webhook imzası doğrulanıyor
- [ ] Kullanıcı input'u DTO ile valide ediliyor
- [ ] `passwordHash`, `taxNumber` hiçbir response'da yok
- [ ] S3 bucket public değil, CloudFront signed URL kullanılıyor
- [ ] Rate limiting Nginx seviyesinde aktif
- [ ] CORS sadece izin verilen origin'lere açık
- [ ] SQL injection: Prisma parametrize sorgular kullanıyor (raw SQL yasak)

---

15. Ajan İçin Görev Başlangıç Protokolü

Bir göreve başlamadan önce şu soruları sor:

1. **Hangi uygulamada çalışıyorum?** (`apps/web`, `apps/api`, `apps/mobile`, `apps/admin`)
2. **Hangi modül etkileniyor?** (auth, products, orders, payments...)
3. **Hangi kullanıcı rolü bu özelliği kullanacak?** (ADMIN, SUPPLIER, BUYER)
4. **Bu bir SSR mi yoksa client-side mı işlem?** (web için kritik)
5. **Async bir işlem var mı?** (varsa BullMQ queue'ya al)
6. **Veritabanı değişikliği var mı?** (varsa Prisma migration oluştur)
7. **Meilisearch senkronizasyonu gerekiyor mu?** (ürün/stok değişikliği için)

Yeni bir özellik yazarken şu sıraya uy:

```
Prisma schema → Migration → DTO → Repository → Service → Controller → Frontend
hook → UI
```

---

16. Proje Büyüdükçe Dikkat Edilecekler

- **Monorepo'ya yeni app ekleme:** Önce `packages/types`'a gerekli tipleri ekle, sonra app'i kur.
- **Yeni bir dış servis eklendiğinde:** `apps/api/src/config/` altına config dosyası ekle, sırrı `.env`'e koy.
- **Performans sorunu yaşandığında:** Önce Redis cache'e bak, sonra Prisma sorgularına `select` ekle, en son index'e bak.
- **Bu dosyayı güncelle:** Mimari bir karar değiştiğinde veya yeni bir teknoloji eklendiğinde `MANA.md`'yi güncelle.

Bu belge ToptanNext projesinin yaşayan mimarisidir. Her önemli kararı buraya yansıt.